
习题课五

批处理、流水、多机作业调度



目录

一、题目汇总

二、多机调度问题

题目汇总

6.9 批处理作业调度

给定 n 个作业的集合 $J = \{J_1, J_2, \dots, J_n\}$ 。每个作业 J_i 都有两项任务要分别在 2 台机器上完成。每个作业必须先由机器 1 处理, 再由机器 2 处理。作业 J_i 需要机器 j 的处理时间为 t_{ji} ($i=1, 2, \dots, n; j=1, 2$)。对于一个确定的作业调度, 设 F_{ji} 是作业 i 在机器 j 上完成处理的时间, 则所有作业在机器 2 上完成处理的时间和 $f = \sum_{i=1}^n F_{2i}$ 称为该作业调度的完成时间和。批处理

作业调度问题要求对于给定的 n 个作业, 制定最佳作业调度方案, 使其完成时间和达到最小。

可用优先队列式分支限界法解此问题。由于要从 n 个作业的所有排列中找出有最小完成时间和的作业调度, 因此批处理作业调度问题的解空间树是一棵排列树。对于批处理作业调度问题, 可以证明存在最佳作业调度使得在机器 1 和机器 2 上作业以相同次序完成。

3.9 流水作业调度

n 个作业 $\{1, 2, \dots, n\}$ 要在由 2 台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工, 然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i

071

和 b_i ($1 \leq i \leq n$)。流水作业调度问题要求确定这 n 个作业的最优加工顺序, 使得从第一个作业在机器 M_1 上开始加工, 到最后一个作业在机器 M_2 上加工完成所需的时间最少。

直观上, 一个最优调度应使机器 M_1 没有空闲时间, 且机器 M_2 的空闲时间最少。在一般情况下, 机器 M_2 上会有机器空闲和作业积压两种情况。

设全部作业的集合为 $N = \{1, 2, \dots, n\}$ 。 $S \subseteq N$ 是 N 的作业子集。在一般情况下, 机器 M_1 开始加工 S 中作业时, 机器 M_2 还在加工其他作业, 要等时间 t 后才可利用。将这种情况下完成 S 中作业所需的最短时间记为 $T(S, t)$ 。流水作业调度问题的最优值为 $T(N, 0)$ 。

4.7 多机调度问题

设有 n 个独立的作业 $\{1, 2, \dots, n\}$, 由 m 台相同的机器进行加工处理。作业 i 所需的处理时间为 t_i 。现约定, 任何作业可以在任何一台机器上加工处理, 但未完工前不允许中断处理, 任何作业不能拆分成更小的子作业。

多机调度问题要求给出一种作业调度方案, 使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

这个问题是一个 NP 完全问题, 到目前为止还没有有效的解法。有时, 这类问题用贪心选择策略可以设计出较好的近似算法。

111

5.3 批处理作业调度

1. 问题描述

给定 n 个作业的集合 $J = \{J_1, J_2, \dots, J_n\}$ 。每个作业 J_i 都有两项任务分别在两台机器上完成。每个作业必须先由机器 1 处理, 再由机器 2 处理。作业 J_i 需要机器 j 的处理时间为 t_{ji} ($i=1, 2, \dots, n; j=1, 2$)。对于一个确定的作业调度, 设 F_{ji} 是作业 i 在机器 j 上完成处理的时间, 则所有作业在机器 2 上完成处理的时间和 $f = \sum_{i=1}^n F_{2i}$ 称为该作业调度的完成时间和。

批处理作业调度问题要求, 对于给定的 n 个作业, 制定最佳作业调度方案, 使其完成时间和达到最小。

批处理作业调度问题的一个常见例子是, 在计算机系统中完成一批 n 个作业, 每个作业都要先完成计算, 然后将计算结果打印输出。计算任务由计算机的中央处理器完成, 打印输出任务由打印机完成。在这种情形下, 计算机的中央处理器是机器 1, 打印机是机器 2。

目录

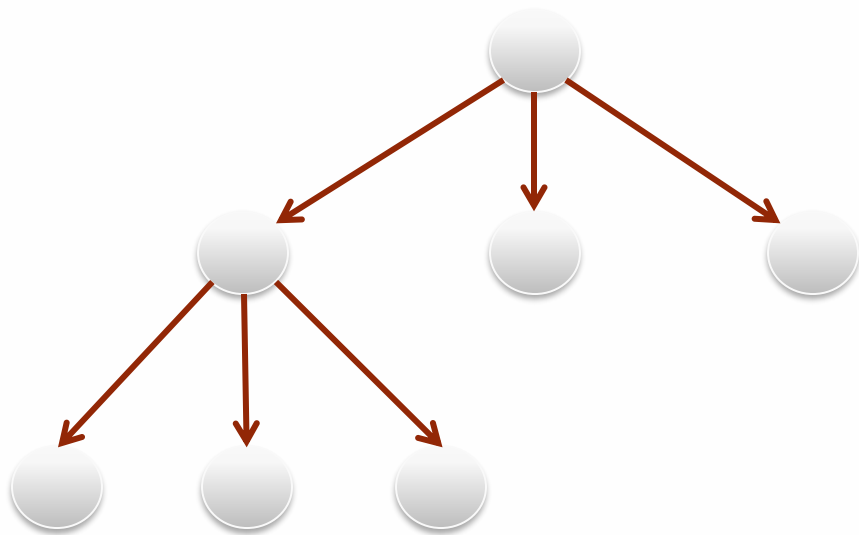
一、题目汇总

二、多机调度问题

多机调度问题

就是把 m 个数字分成 n 组，最大的那一组的和要最小。

暴力：回溯，每次枚举一个任务放到第几个机器上面



对于稍大一些的数据，会超时。

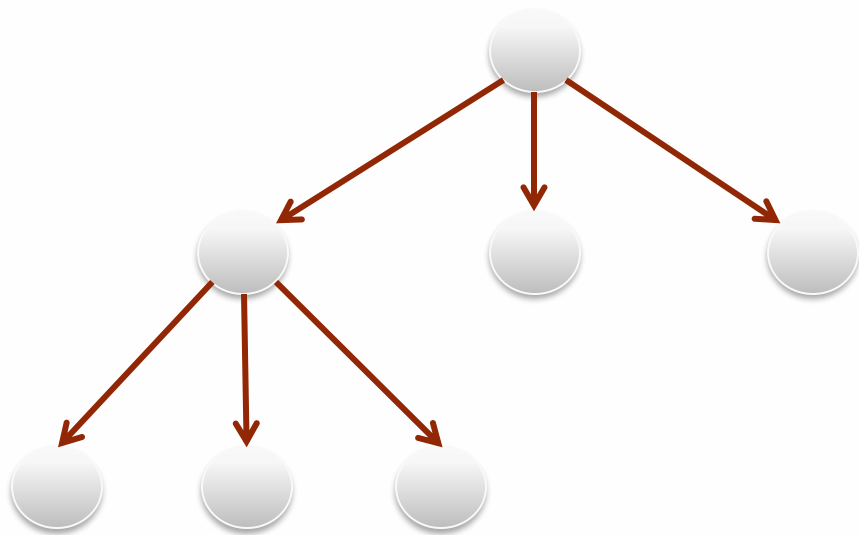
```
tasks = [1, 2, 3, 4, 5, 6, 7, 8, 12, 23, 34, 45]
n = len(tasks)
m = 4# machine
```

```
tasks = [1, 2, 3, 4, 5, 6, 7, 8]
n = len(tasks)
m = 3 # machine
groups = [[] for _ in range(m)]
groups_sum = [0] * m
ans = 1e12
ans_case = None
def dfs(i: int):
    global ans, ans_case
    if i == n:
        if max(groups_sum) < ans:
            ans_case = copy.deepcopy(groups)
            ans = max(groups_sum)
        return
    for j in range(m):
        t = tasks[i]
        groups[j].append(t)
        groups_sum[j] += t
        dfs(i + 1)
        groups[j].pop()
        groups_sum[j] -= t
    return
dfs(0)
print(ans_case, ans)
```

多机调度问题

就是把 m 个数字分成 n 组，最大的那一组的和要最小。

暴力：回溯，每次枚举一个任务放到第几个机器上面



一条路径就是一个可行解。

剪枝操作2：对于已经大于当前，最优答案的，剪去。

```
def dfs(i: int):
    global ans, ans_case
    if i == n:
        if max(groups_sum) < ans:
            ans_case = copy.deepcopy(groups)
            ans = max(groups_sum)
        return

    for j in range(m):
        t = tasks[i]
        groups[j].append(t)
        groups_sum[j] += t
        if groups_sum[j] < ans: # 剪枝1
            dfs(i + 1)
        groups[j].pop()
        groups_sum[j] -= t

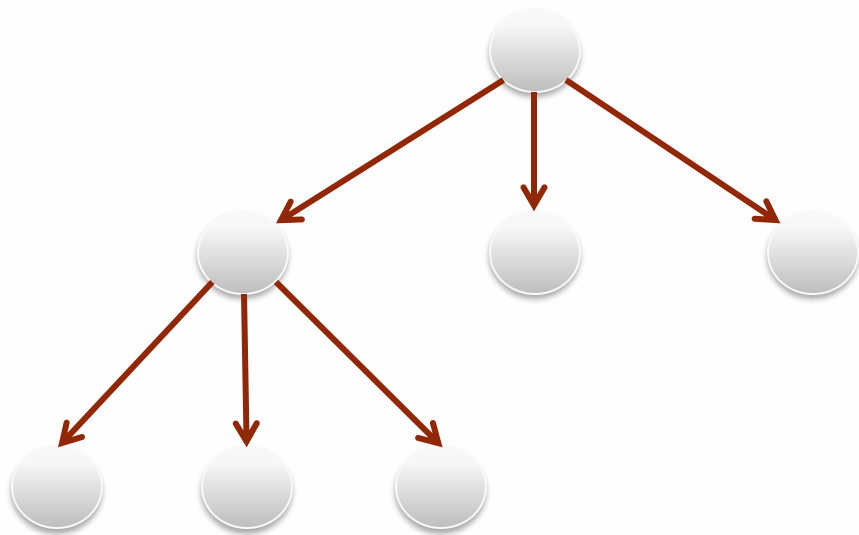
    return

dfs(0)
```

多机调度问题

就是把m个数字分成n组，最大的那一组的和要最小。

暴力：回溯，每次枚举一个任务放到第几个机器上面



剪枝操作2：自反等价，计算结果完全一样，剪去。

```
def dfs(i: int):
    global ans, ans_case
    if i == n:
        if max(groups_sum) < ans:
            ans_case = copy.deepcopy(groups)
            ans = max(groups_sum)
        return
    for j in range(m):
        t = tasks[i]
        if j >= 1 and groups_sum[j - 1] == groups_sum[j]: # 剪枝2
            continue
        groups[j].append(t)
        groups_sum[j] += t
        if groups_sum[j] < ans: # 剪枝1
            dfs(i + 1)
        groups[j].pop()
        groups_sum[j] -= t
    return

dfs(0)
```

多机调度问题

1.py 小数据量，纯回溯

2.py 大数据量，纯回溯

3.py 大数据量，加入剪枝**1**

4.py 大数据量，加入剪枝**1**和剪枝**2**

5.py 大数据量，排序以加速命中剪枝

```
> python 1.py
[[1, 2, 3, 6], [4, 8], [5, 7]] 12 cost 0.003 seconds
> python 2.py
[[1, 2, 3, 4, 5, 6, 7, 8], [12, 23], [34], [45]] 45 cost 4.919 seconds
> python 3.py
[[1, 2, 3, 4, 5, 6, 7, 8], [12, 23], [34], [45]] 45 cost 1.087 seconds
> python 4.py
[[1, 2, 3, 4, 5, 6, 7, 8], [12, 23], [34], [45]] 45 cost 0.055 seconds
> python 5.py
[[1, 2, 3, 4, 5, 6, 7, 8], [12, 23], [34], [45]] 45 cost 0.054 seconds
```


多机调度问题

前面的算法都是回溯，剪枝完之后时间复杂度仍然很高，在不追求精确解而需要快速得到近似解的条件下，可以使用贪心算法。

最常用的贪心策略是"最长处理时间优先"
(LPT, Longest Processing Time)
它具有理论保证的近似比。

对于 m 台机器，LPT算法的近似比是
 $(4/3 - 1/(3m))$

```
def lpt_schedule(tasks, m):
    """
    LPT (Longest Processing Time) 贪心算法
    tasks: 任务时间列表
    m: 机器数量
    """
    # 按任务处理时间降序排序
    sorted_tasks = sorted(tasks, reverse=True)

    # 初始化每台机器的工作时间
    machines = [0] * m
    # 记录每台机器分配的任务
    assignments = [[] for _ in range(m)]

    # 依次将任务分配给当前负载最小的机器
    for task in sorted_tasks:
        # 找到当前负载最小的机器
        min_load_machine = machines.index(min(machines))
        # 分配任务
        machines[min_load_machine] += task
        assignments[min_load_machine].append(task)

    return assignments, max(machines)

if __name__ == "__main__":
    # 使用与回溯算法相同的测试用例
    tasks = [1, 2, 3, 4, 5, 6, 7, 8, 12, 23, 34, 45]
    m = 4 # 机器数量

    start = time.time()
    assignments, makespan = lpt_schedule(tasks, m)
    end = time.time()

    print(assignments, f"cost {round(end - start, 6)} seconds")
```